Variantes de la lista enlazada

Lo que compra cada puntero, y merge sort

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Construir la lista doblemente enlazada y la circular, y decir qué operación baja de costo y qué se paga a cambio en cada una (OA2, OA3).
- Explicar por qué llegar a la posición p por el lado más corto parte el trabajo a la mitad sin cambiar la cota $\Theta(n)$ (OA1).
- Ordenar una lista enlazada partiéndola, ordenando cada mitad y mezclando, y escribir la recurrencia que da $\Theta(n \log n)$ (OA1, OA5).
- Decir por qué la mezcla sobre nodos ocupa $\Theta(1)$ de espacio aparte y sobre un arreglo ocupa $\Theta(n)$ (OA1, OA3).

Contenido I

1 Lo que compró el puntero al último

2 El nodo que mira atrás

3 La cadena que se cierra

4 La mitad del camino

5 Ordenar partiendo y mezclando

6 Lo que cobra cada implementación

7 Ejercicios

Contenido I

1 Lo que compró el puntero al último

2 El nodo que mira atrás

3 La cadena que se cierra

4 La mitad del camino

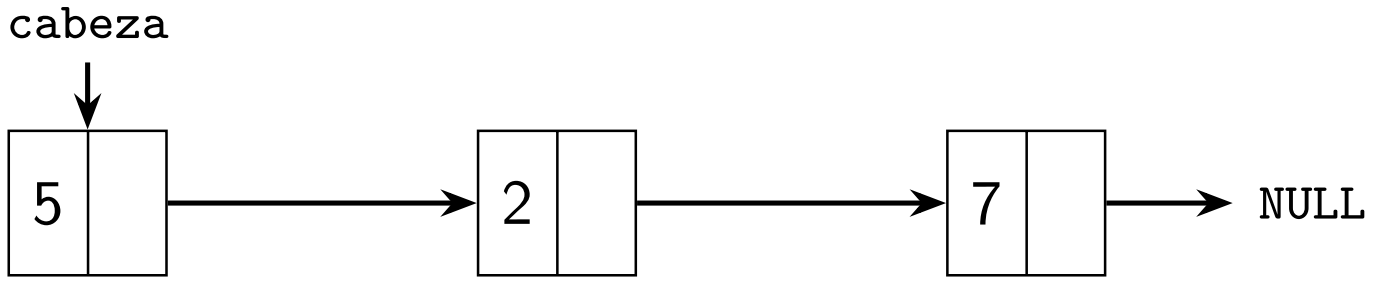
5 Ordenar partiendo y mezclando

6 Lo que cobra cada implementación

7 Ejercicios

Dónde quedó la lista enlazada I

La cadena arranca en `cabeza` y termina en `NULL`. Insertar al frente son dos escrituras de puntero; llegar a la posición p son p pasos.



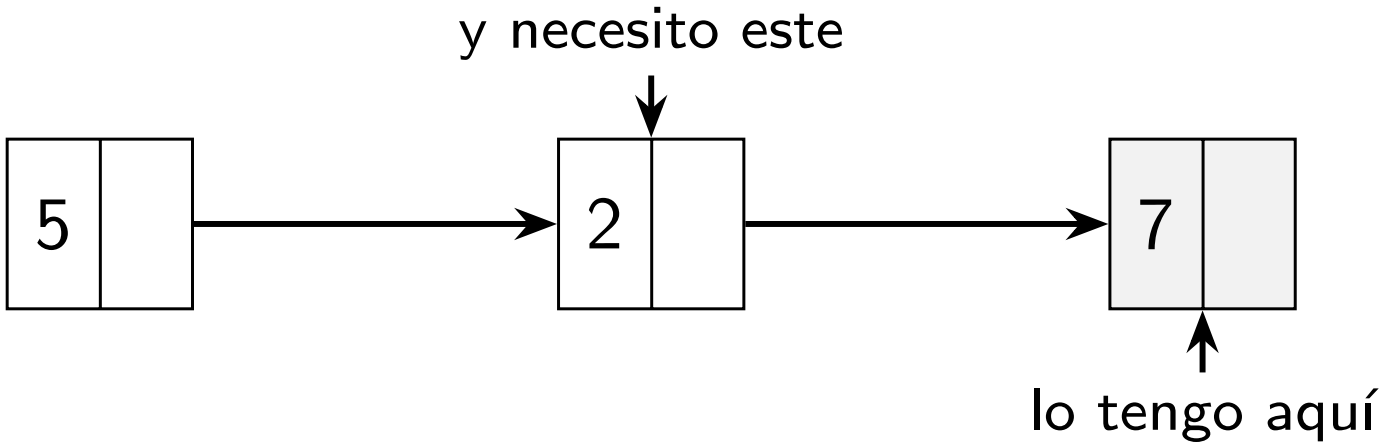
Guardar un puntero al último bajó `agregar` de $\Theta(n)$ a $\Theta(1)$, y cobró su precio: `eliminar` ganó dos casos, borrar el último y vaciar la lista.

Pregunta

Supongamos que ya tengo en la mano un puntero al nodo que quiero borrar. En esta lista, ¿cuánto cuesta borrarlo?

Cuesta toda la lista I

Para sacar un nodo de la cadena hay que reescribir el campo `siguiente` *del anterior*, y el nodo no sabe quién es su anterior. Hay que buscarlo desde la cabeza: $\Theta(n)$, aunque el empalme en sí sea $\Theta(1)$.



Esa es la operación que paga la siguiente variante.

Contenido I

1 Lo que compró el puntero al último

2 El nodo que mira atrás

3 La cadena que se cierra

4 La mitad del camino

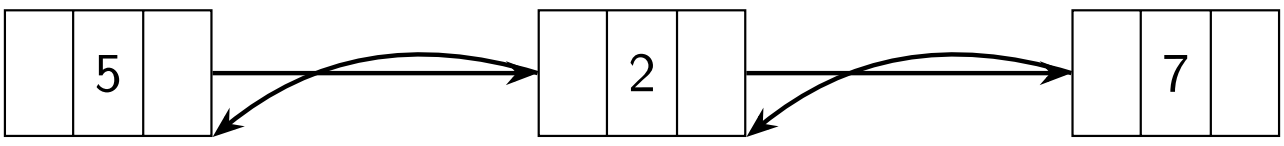
5 Ordenar partiendo y mezclando

6 Lo que cobra cada implementación

7 Ejercicios

Un campo más I

```
struct Nodo {
  Elemento dato;
  Nodo *anterior;
  Nodo *siguiente;
};
```



`anterior` | `dato` | `siguiente`

La lista guarda además el último, que ahora sale gratis mantener porque los enlaces van en los dos sentidos.

Desenlazar sin recorrer I

```
// Saca el nodo de la cadena. No recorre nada: por eso se
↪  guarda anterior.
void desenlazar(Nodo *x) {
  if (x->anterior == NULL) {
    cabeza = x->siguiente;
  } else {
    x->anterior->siguiente = x->siguiente;
  }
  if (x->siguiente == NULL) {
    ultimo = x->anterior;
  } else {
    x->siguiente->anterior = x->anterior;
  }
}
```

Cuatro escrituras de puntero en el peor caso y ningún recorrido: $\Theta(1)$. Los dos `if` son los extremos de la cadena, donde del otro lado no hay nodo sino `NULL`.

Desenlazar sin recorrer II

```
void eliminar(int p) {
  assert(0 <= p && p < n);
  Nodo *muerto = nodoEn(p);
  desenlazar(muerto);
  delete muerto;
  n = n - 1;
}
```

Lo que no cambió

`eliminar(p)` sigue costando $\Theta(p)$, porque `nodoEn` tiene que caminar. Lo que bajó a $\Theta(1)$ es borrar un nodo *que ya se tiene*, y eso es justo lo que pasa al recorrer la lista borrando, o al guardar punteros a elementos para quitarlos después.

Lo que cobra el campo nuevo I

`insertar` ahora mantiene dos enlaces por nodo y tiene cuatro casos: lista vacía, al frente, al final y en medio.

```
if (n == 0) {
  nuevo->anterior = NULL;
  nuevo->siguiente = NULL;
  cabeza = nuevo;
  ultimo = nuevo;
} else if (p == 0) {
  nuevo->anterior = NULL;
  nuevo->siguiente = cabeza;
  cabeza->anterior = nuevo;
  cabeza = nuevo;
}
```

Lo que cobra el campo nuevo II

```
} else if (p == n) {
  nuevo->anterior = ultimo;
  nuevo->siguiente = NULL;
  ultimo->siguiente = nuevo;
  ultimo = nuevo;
} else {
  Nodo *x = nodoEn(p);
  nuevo->anterior = x->anterior;
  nuevo->siguiente = x;
  x->anterior->siguiente = nuevo;
  x->anterior = nuevo;
}
```

Cada caso existe porque en algún extremo falta un vecino. Ese es el problema que resuelve la variante siguiente.

Contenido I

1 Lo que compró el puntero al último

2 El nodo que mira atrás

3 La cadena que se cierra

4 La mitad del camino

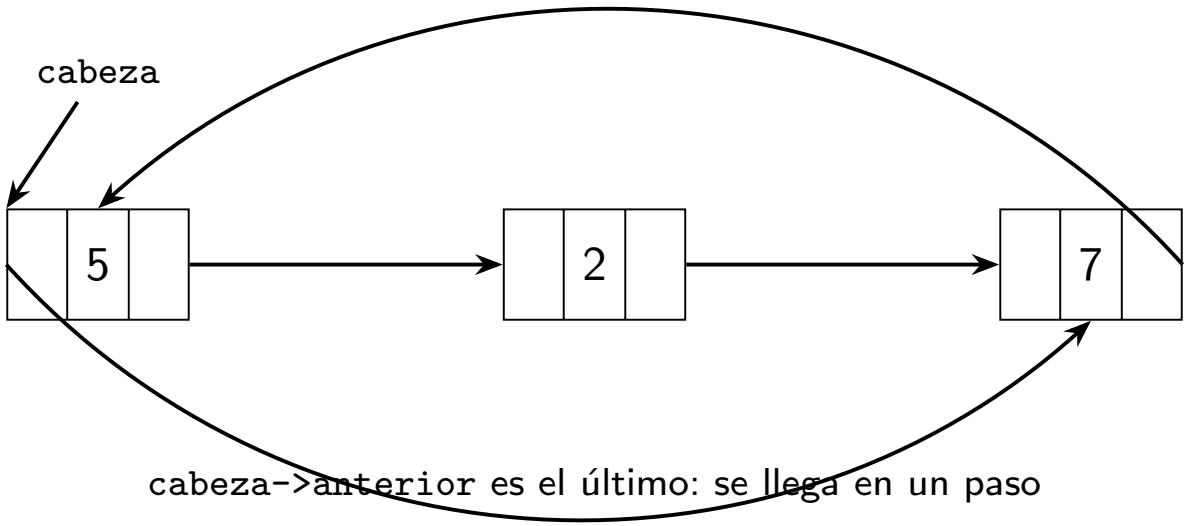
5 Ordenar partiendo y mezclando

6 Lo que cobra cada implementación

7 Ejercicios

Sin `NULL` no hay extremos I

Si el último apunta al primero y el primero al último, la cadena no tiene puntas. Desaparece `NULL` y con él los casos que existían solo para los extremos.



La clase guarda `cabeza` y `n`, y nada más. El último ya no hay que guardarlo aparte.

Insertar, con dos casos en vez de cuatro I

```
void insertar(int p, Elemento e) {
  assert(0 <= p && p <= n);
  Nodo *nuevo = new Nodo;
  nuevo->dato = e;
  if (n == 0) {
    nuevo->siguiente = nuevo;
    nuevo->anterior = nuevo;
    cabeza = nuevo;
  }
```

Un nodo solo cierra el círculo consigo mismo. Todo lo demás cae en la otra rama:

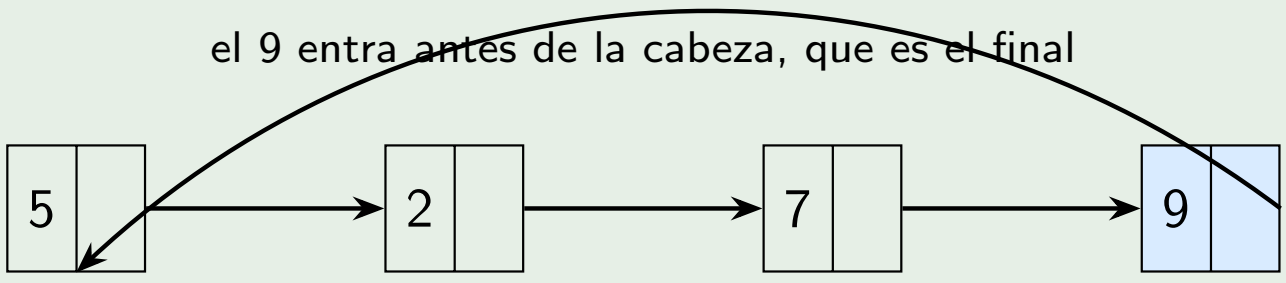
Insertar, con dos casos en vez de cuatro II

```
  } else {
    // se empalma antes de x; con p == n, x es la cabeza y
    ↪  queda al final
    Nodo *x = cabeza;
    if (p < n) {
      x = nodoEn(p);
    }
    nuevo->siguiente = x;
    nuevo->anterior = x->anterior;
    x->anterior->siguiente = nuevo;
    x->anterior = nuevo;
    if (p == 0) {
      cabeza = nuevo;
    }
  }
  n = n + 1;
}
```

Insertar, con dos casos en vez de cuatro III

Las cuatro líneas del empalme sirven para cualquier posición, porque `x->anterior` siempre existe. Insertar al final es insertar antes de la cabeza: en un círculo son el mismo lugar.

$p = n$ sobre $\langle 5, 2, 7 \rangle$



El caso que sí aparece

Con un solo nodo, ese nodo es su propio siguiente y su propio anterior. Al borrarlo no hay a quién reenlazar: la lista queda vacía y `cabeza` vuelve a `NULL`.

Contenido I

1 Lo que compró el puntero al último

2 El nodo que mira atrás

3 La cadena que se cierra

4 La mitad del camino

5 Ordenar partiendo y mezclando

6 Lo que cobra cada implementación

7 Ejercicios

Una pregunta de pasos I

La lista circular doble tiene $n = 10$ elementos y se pide la posición 8.

Pregunta

Caminando desde la cabeza hacia adelante son 8 pasos. Con los enlaces hacia atrás y sabiendo que hay 10 elementos, ¿cuántos pasos hacen falta?

Dos

`cabeza->anterior` es la posición 9 y `cabeza->anterior->anterior` es la 8. La posición 8 está a 8 pasos de la cabeza yendo hacia adelante y a 2 yendo hacia atrás.

Caminar por el lado más corto I

```
Nodo *nodoEn(int p) {
  Nodo *actual = cabeza;
  if (p <= n / 2) {
    int i = 0;
    while (i < p) {
      actual = actual->siguiente;
      i = i + 1;
    }
  } else {
    int i = n;
    while (i > p) {
      actual = actual->anterior;
      i = i - 1;
    }
  }
  return actual;
}
```

Caminar por el lado más corto II

Hacen falta las dos cosas que distinguen a esta variante: el enlace `anterior` y el contador `n`. Sin el contador no se sabe cuál lado es el corto.

Caminar por el lado más corto III

Pasos medidos, con $n = 10$

p	0	1	2	3	4	5	6	7	8	9
de frente	0	1	2	3	4	5	6	7	8	9
por el lado corto	0	1	2	3	4	5	4	3	2	1

Recorriendo todas las posiciones una vez:

n	de frente	por el lado corto
10	45	25
100	4 950	2 500
1000	499 500	250 000

La cota no se movió I

Teorema

Sea pasos(p) el número de pasos que da `nodoEn(p)` en la lista circular doble, con $0 \leq p < n$. Entonces pasos(p) $\leq \lceil n/2 \rceil$ para todo p.

La cota no se movió II

Demostración

Se procede de forma directa, separando los dos caminos que toma el código.

Desarrollo. Si $p \leq \lfloor n/2 \rfloor$, el ciclo avanza desde $i = 0$ hasta $i = p$ y da exactamente p pasos, de modo que $pasos(p) = p \leq \lfloor n/2 \rfloor \leq \lceil n/2 \rceil$.

Si $p > \lfloor n/2 \rfloor$, el ciclo retrocede desde $i = n$ hasta $i = p$ y da $n - p$ pasos. De $p > \lfloor n/2 \rfloor$ se sigue $p \geq \lfloor n/2 \rfloor + 1$, y por lo tanto $n - p \leq n - \lfloor n/2 \rfloor - 1 = \lceil n/2 \rceil - 1 < \lceil n/2 \rceil$.

Conclusión. En los dos casos $pasos(p) \leq \lceil n/2 \rceil$, de donde $pasos(p) \in O(n)$ con testigos $c = 1$ y $k = 1$.

La cota no se movió III

Lo que esto no dice

El peor caso pasó de $n - 1$ pasos a $\lceil n/2 \rceil$: la mitad del trabajo. Pero $\Theta(n/2) = \Theta(n)$, porque la notación asintótica ignora el factor constante. La cota es la misma y el programa corre al doble de rápido en esa operación.

Las dos afirmaciones conviven: la cota clasifica el crecimiento, no mide el tiempo. Duplicar n sigue duplicando el trabajo en las dos versiones; lo que cambió es cuánto trabajo hay para cada n.

Contenido I

1 Lo que compró el puntero al último

2 El nodo que mira atrás

3 La cadena que se cierra

4 La mitad del camino

5 Ordenar partiendo y mezclando

6 Lo que cobra cada implementación

7 Ejercicios

Lo que frenaba a insertion sort I

Insertion sort acomoda un elemento a la vez en el trozo ya ordenado. Sobre la enlazada el corrimiento desaparece —meter un nodo en su lugar es un empalme—, pero encontrar ese lugar exige caminar, y la vuelta j puede recorrer j nodos: $\Theta(n^2)$ en el peor caso.

La otra idea

En vez de acomodar un elemento contra todos los anteriores, partir la lista en dos, ordenar cada mitad y combinar las dos mitades ya ordenadas. Combinar dos listas ordenadas es barato: se comparan las dos cabezas y se toma la menor.

Tres pasos: **dividir** la lista en dos mitades, **conquistar** ordenando cada una con el mismo procedimiento, y **combinar** mezclando las dos.

Dividir: dos punteros a distinta velocidad I

La lista no tiene índices, así que la mitad no se calcula: se encuentra. Un puntero avanza de a uno y otro de a dos; cuando el rápido llega al final, el lento va por la mitad.

```
Nodo *partir(Nodo *cabeza) {
  Nodo *lento = cabeza;
  Nodo *rapido = cabeza->siguiente;
  while (rapido != NULL && rapido->siguiente != NULL) {
    lento = lento->siguiente;
    rapido = rapido->siguiente->siguiente;
  }
  Nodo *segunda = lento->siguiente;
  lento->siguiente = NULL;
  return segunda;
}
```

Dividir: dos punteros a distinta velocidad II

Traza sobre $\langle 6, 2, 9, 4, 1, 7 \rangle$

vuelta	lento señala	rápido señala
inicio	6	2
1	2	4
2	9	7
fin	9	`NULL` tras 7

Se corta después del 9: queda $\langle 6, 2, 9 \rangle$ y $\langle 4, 1, 7 \rangle$.

Un solo recorrido y ningún nodo nuevo: $\Theta(n)$ en tiempo y $\Theta(1)$ en espacio.

Combinar: empalmar, no copiar I

```
Nodo *mezclar(Nodo *a, Nodo *b) {
  Nodo guia;
  guia.dato = 0;
  guia.siguiente = NULL;
  Nodo *cola = &guia;
  while (a != NULL && b != NULL) {
    if (a->dato <= b->dato) {
      cola->siguiente = a;
      a = a->siguiente;
    } else {
      cola->siguiente = b;
      b = b->siguiente;
    }
    cola = cola->siguiente;
  }
```

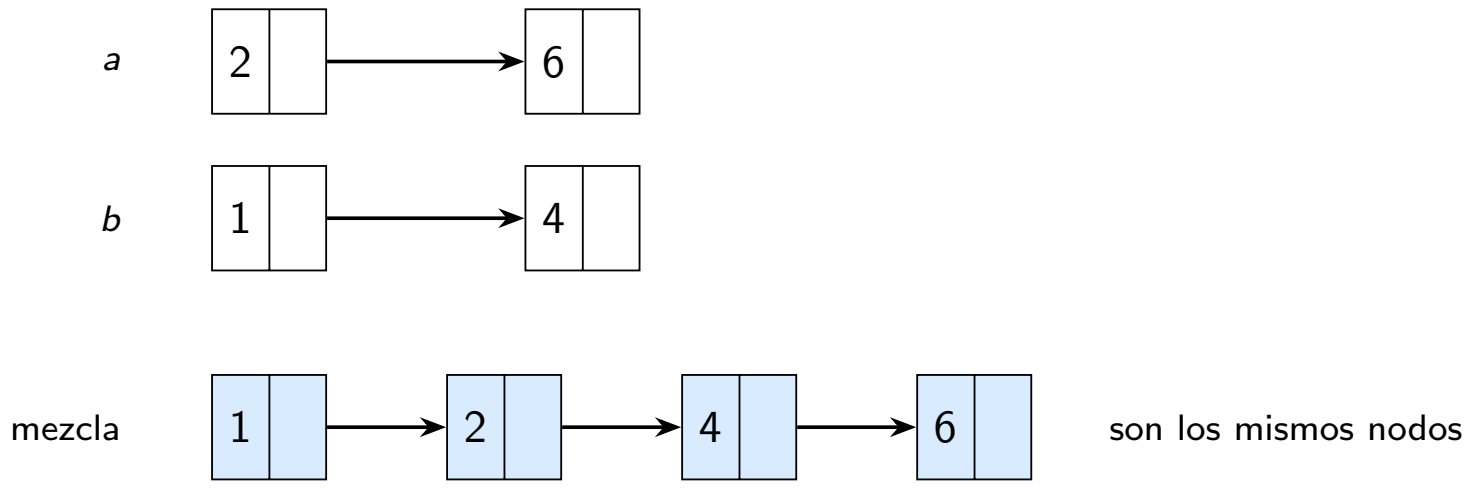
Cuando una de las dos se acaba, lo que queda de la otra ya está ordenado y se cuelga entero:

Combinar: empalmar, no copiar II

```
  if (a != NULL) {
    cola->siguiente = a;
  } else {
    cola->siguiente = b;
  }
  return guia.siguiente;
}
```

Combinar: empalmar, no copiar III

El nodo `guia` vive en la pila y no guarda ningún elemento: está para que la primera vuelta escriba `cola->siguiente` como todas las demás, sin un caso aparte para la lista vacía. Al final se devuelve lo que quedó colgando de él.



No se reserva ni un nodo y no se copia ni un dato: solo se reescribe el campo `siguiente`. Por eso la mezcla ocupa $\Theta(1)$ de espacio aparte.

Combinar: empalmar, no copiar IV

Lo que sostiene el ciclo

Sea M la cadena que cuelga de `guia` al empezar una vuelta, con $|M|$ nodos.

$I_1 : \forall i, j,\ 0 \le i < j < |M| :\ M[i] \le M[j]$

$I_2 : |M| > 0 \ \Rightarrow\ M[|M|-1] \le \mathrm{dato}(a) \ \wedge\ M[|M|-1] \le \mathrm{dato}(b)$

$I_3 : M \cup \mathrm{resto}(a) \cup \mathrm{resto}(b) = A \cup B$ como multiconjuntos

I_1 dice que lo mezclado está en orden; I_2, que lo que falta no es menor que lo ya puesto, que es lo que permite agregar al final sin revisar atrás; I_3, que no se pierde ni se inventa ningún nodo.

El algoritmo completo I

```
// Una lista de cero o un nodo ya esta ordenada.
Nodo *ordenar(Nodo *cabeza) {
  if (cabeza != NULL && cabeza->siguiente != NULL) {
    Nodo *segunda = partir(cabeza);
    cabeza = mezclar(ordenar(cabeza), ordenar(segunda));
  }
  return cabeza;
}
```

Sobre $\langle 6, 2, 9, 4, 1, 7, 3, 8 \rangle$

```
antes:    6 2 9 4 1 7 3 8
despues: 1 2 3 4 6 7 8 9
```

17 comparaciones.

El algoritmo completo II

El costo

Partir recorre la lista una vez y mezclar también, así que el trabajo fuera de las llamadas es $\Theta(n)$. Cada llamada se parte en dos de tamaño $n/2$:

$$T(n) = 2\,T(n/2) + \Theta(n), \qquad T(1) = \Theta(1)$$

El árbol de llamadas tiene $\log_2 n$ niveles y cada nivel hace $\Theta(n)$ de trabajo en total, de donde $T(n) = \Theta(n \log n)$. No hay mejor ni peor caso: la lista se parte por la mitad sin mirar los datos.

	Tiempo	Espacio aparte de la lista
`partir`	$\Theta(n)$	$\Theta(1)$
`mezclar`	$\Theta(n_a + n_b)$	$\Theta(1)$
`ordenar`	$\Theta(n \log n)$	$\Theta(\log n)$, la pila de llamadas

Las ocho, paso a paso I

Sobre $\langle 6, 2, 9, 4, 1, 7, 3, 8 \rangle$, bajando hasta listas de un nodo y subiendo mezclando.

Nivel	Al bajar, `partir` deja	Al subir, `mezclar` devuelve
0	$\langle 6\,2\,9\,4\,1\,7\,3\,8 \rangle$	$\langle 1\,2\,3\,4\,6\,7\,8\,9 \rangle$
1	$\langle 6\,2\,9\,4 \rangle$ $\langle 1\,7\,3\,8 \rangle$	$\langle 2\,4\,6\,9 \rangle$ $\langle 1\,3\,7\,8 \rangle$
2	$\langle 6\,2 \rangle$ $\langle 9\,4 \rangle$ $\langle 1\,7 \rangle$ $\langle 3\,8 \rangle$	$\langle 2\,6 \rangle$ $\langle 4\,9 \rangle$ $\langle 1\,7 \rangle$ $\langle 3\,8 \rangle$
3	$\langle 6 \rangle \langle 2 \rangle \langle 9 \rangle \langle 4 \rangle \langle 1 \rangle \langle 7 \rangle \langle 3 \rangle \langle 8 \rangle$	ya están ordenadas

Las ocho, paso a paso II

Las comparaciones, por nivel

Nivel	Comparaciones	De dónde salen
2	4	cuatro mezclas de uno contra uno
1	6	dos mezclas de dos contra dos
0	7	una mezcla de cuatro contra cuatro
	17	

Cada nivel toca los 8 nodos y hay $\log_2 8 = 3$ niveles de mezcla. Ahí está el $n \log n$: no en una vuelta larga, sino en pocas vueltas sobre todo.

Sobre el arreglo cuesta más espacio I

La mezcla sobre un arreglo no puede empalmar: para intercalar dos mitades en el sitio habría que correr elementos, así que se escribe en un arreglo auxiliar y después se copia de vuelta. Eso es $\Theta(n)$ de espacio extra.

	Sobre la enlazada	Sobre el arreglo
tiempo	$\Theta(n \log n)$	$\Theta(n \log n)$
espacio de la mezcla	$\Theta(1)$	$\Theta(n)$
qué se mueve	un puntero por nodo	cada elemento

Sobre el arreglo cada elemento se escribe dos veces por nivel, al auxiliar y de vuelta. Es el único punto del curso donde la enlazada le gana al arreglo en un algoritmo completo, y gana justo por lo que la hacía cara: que los elementos no están pegados y moverlos es reescribir una dirección.

Los dos algoritmos sobre la misma entrada I

n	insertion sort	merge sort	veces más
16	82	46	1
128	4 066	748	5
1024	258 018	8 181	31
4096	4 088 802	37 214	109

Comparaciones contadas sobre la misma permutación.

Con 16 elementos casi da lo mismo

Merge sort hace 46 comparaciones contra 82: ni el doble. La ventaja aparece cuando n crece, y por eso con listas pequeñas el algoritmo simple compite. Una cota asintótica dice qué pasa al crecer, no cuál conviene para un tamaño fijo.

Contenido I

1 Lo que compró el puntero al último

2 El nodo que mira atrás

3 La cadena que se cierra

4 La mitad del camino

5 Ordenar partiendo y mezclando

6 Lo que cobra cada implementación

7 Ejercicios

Las cuatro variantes del mismo contrato I

Operación	Simple	Doble	Circular	Circular, lado corto
`obtener(p)`, `asignar(p, e)`	$\Theta(p)$	$\Theta(p)$	$\Theta(p)$	$\Theta(\text{mín}(p, n-p))$
`insertar(0, e)`	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
`insertar(p, e)`, `eliminar(p)`	$\Theta(p)$	$\Theta(p)$	$\Theta(p)$	$\Theta(\text{mín}(p, n-p))$
`agregar(e)`	$\Theta(n)$, o $\Theta(1)$ con `ultimo`	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
desenlazar un nodo que ya se tiene	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
llegar al último	$\Theta(n)$, o $\Theta(1)$ con `ultimo`	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$
recorrer hacia atrás	no se puede	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
casos de `insertar`	2	4	2	2
espacio por nodo	dato y 1 puntero	dato y 2	dato y 2	dato y 2
espacio de la lista	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$

Las cuatro variantes del mismo contrato II

Por qué cuesta lo que cuesta

- Todas caminan para llegar a una posición, porque ninguna calcula direcciones: ese es el precio de no estar en casillas consecutivas.
- El campo `anterior` convierte el desenlace en cuatro escrituras fijas, y a cambio cada modificación mantiene dos enlaces en vez de uno.
- Cerrar el círculo borra los extremos, y con ellos los casos que existían solo porque faltaba un vecino.
- El contador deja escoger el lado: la cota sigue siendo $\Theta(n)$ y el peor caso baja de $n-1$ pasos a $\lceil n/2 \rceil$.

La estructura se escoge por las operaciones que el programa repite, no por cuál se ve más completa.

Para el juez: UVa 11462 — Age Sort I

UVa 11462 — Age Sort,
`https://vjudge.net/problem/UVA-11462`
Envío: `https://vjudge.net/problem/UVA-11462`

Qué pide

Varios casos. Cada uno trae n edades de habitantes de una ciudad y hay que imprimirlas en orden no decreciente, separadas por espacio, una línea por caso. El archivo termina con $n = 0$.

Entrada, salida y límites

$0 < n \leq 2\,000\,000$ y cada edad está entre 1 y 99. La suma de todos los n del archivo cabe en memoria, pero no sobra: el enunciado advierte que la entrada es grande.

Para el juez: UVa 11462 — Age Sort II

Cómo atacarlo con lo de hoy

Merge sort ordena en $\Theta(n \log n)$: con $n = 2$ millones son unos $2 \cdot 10^6 \times 21 \approx 4{,}2 \cdot 10^7$ comparaciones, que entran en el tiempo. Sirve, y es la solución que se escribe con lo visto.

Las trampas

- El cuello de botella no es ordenar: es leer. Los dos millones de enteros se leen con `scanf` y se escriben con `printf`.
- Una lista enlazada de dos millones de nodos pide dos millones de `new`: el tiempo de reservar y la memoria del puntero por nodo pesan. Aquí el arreglo gana.
- Las edades van de 1 a 99. Esa cota sobre los *valores*, no sobre la cantidad, permite ordenar sin comparar y en $\Theta(n)$. Vale la pena averiguar cómo antes de la sesión de tablas hash.

Para el juez: UVa 10810 — Ultra-QuickSort I

UVa 10810 — Ultra-QuickSort,
`https://vjudge.net/problem/UVA-10810`
Envío: `https://vjudge.net/problem/UVA-10810`

Qué pide

Varios casos. Cada uno trae una sucesión de n enteros distintos y hay que imprimir cuántos intercambios de vecinos hacen falta para dejarla en orden ascendente. El archivo termina con $n = 0$, y ese caso no se procesa.

Entrada, salida y límites

$n < 500\,000$, un elemento por línea y $0 \leq a_i \leq 999\,999\,999$. Por cada caso se imprime una línea con el número de intercambios: para $\langle 9, 1, 0, 5, 4 \rangle$ la respuesta es 6, y para $\langle 1, 2, 3 \rangle$ es 0.

Para el juez: UVa 10810 — Ultra-QuickSort II

Cómo atacarlo con lo de hoy

Lo que pide es la cantidad de pares en desorden, que es exactamente el número de corrimientos que haría insertion sort. Contarlos así cuesta $\Theta(n^2)$, y medio millón de elementos no entra. La cuenta cabe dentro de `mezclar` sin tocar su costo: cada vez que gana un elemento de la segunda cadena, ese elemento se adelanta a todos los que siguen pendientes en la primera, y esos son los que se suman. Para saber cuántos son hay que llevar el tamaño de cada mitad; recorrer lo que queda para contarlo en cada paso devuelve el $\Theta(n^2)$ que se estaba evitando.

Para el juez: UVa 10810 — Ultra-QuickSort III

Las trampas

- La respuesta llega hasta $n(n-1)/2$, unos $1{,}2 \cdot 10^{11}$ con medio millón de elementos: no cabe en `int`. El contador va en `long long`, y también lo que devuelve cada llamada recursiva.
- Cuando una de las dos cadenas se acaba y el resto de la otra se cuelga entero, no se suma nada. Los pares que cuentan son los que se formaron al adelantar un elemento de la segunda.
- El contador arranca en cero en cada caso. El ejemplo del enunciado trae dos sucesiones y el segundo caso responde 0.

Contenido I

1 Lo que compró el puntero al último

2 El nodo que mira atrás

3 La cadena que se cierra

4 La mitad del camino

5 Ordenar partiendo y mezclando

6 Lo que cobra cada implementación

7 Ejercicios

Ejercicio en parejas: rotar la circular I

Escriba `rotar(int k)` sobre la lista circular doble: la posición k pasa a ser la cabeza y el orden circular no cambia. Con $\langle 5, 2, 7, 4 \rangle$ y $k = 2$ queda $\langle 7, 4, 5, 2 \rangle$.

Las preguntas

1. ¿Cuántos nodos hay que mover de sitio?
2. ¿Cuánto cuesta con $k = 1$? ¿Y con $k = n - 1$?
3. Escríbalo de modo que nunca dé más de $\lceil n/2 \rceil$ pasos.
4. ¿Se puede hacer lo mismo sobre la lista estática? ¿Qué costaría?

Ejercicio en parejas: rotar la circular II

Lo que debe salir

No se mueve ningún nodo: solo cambia a cuál apunta `cabeza`, porque en un círculo todos los elementos ya están en su lugar relativo. Con $k = 1$ es un paso y con $k = n - 1$ es un paso hacia atrás, así que escogiendo el sentido queda $\Theta(\text{mín}(k,\, n - k))$. Sobre el arreglo hay que correr los n elementos: $\Theta(n)$ y sin manera de evitarlo, porque la posición 0 es una casilla fija.

Ejercicio en parejas: quitar repetidos de una lista ordenada I

La lista enlazada simple viene ordenada y trae repetidos: $\langle 1, 1, 3, 5, 5, 5, 8 \rangle$. Deje una sola copia de cada valor, liberando los nodos que sobran.

Las preguntas

1. ¿Cuántos recorridos hacen falta?
2. ¿Qué puntero hay que guardar antes de cada `delete`, y por qué?
3. ¿Cuántos nodos se liberan en el ejemplo?
4. ¿Cuánto cuesta en tiempo y en espacio aparte de la lista?

Ejercicio en parejas: quitar repetidos de una lista ordenada II

Lo que debe salir

Un solo recorrido. Estando ordenada, los repetidos quedan consecutivos: basta comparar cada nodo con el siguiente y, si son iguales, empalmar sobre el siguiente y liberarlo. Antes del `delete` hay que guardar `siguiente` del nodo que se borra, que es a donde hay que enlazar.
Se liberan tres nodos y queda $\langle 1, 3, 5, 8 \rangle$. Es $\Theta(n)$ en tiempo y $\Theta(1)$ en espacio: los nodos no se copian, se sueltan.

Si no estuviera ordenada

Los repetidos no serían consecutivos y habría que comparar cada nodo con todos los anteriores: $\Theta(n^2)$. Ordenar primero con merge sort y después quitar repetidos cuesta $\Theta(n \log n)$, que es menos.

Propuestos para la casa I

1. **Buscar por el lado corto.** Escriba `buscar(Elemento x)` sobre la circular doble recorriendo desde los dos extremos a la vez. ¿Cuántas comparaciones hace en el peor caso?
2. **La trampa de partir.** ¿Qué pasa si se le pasa una lista circular a `partir`? Siga el ciclo del puntero rápido y diga en qué condición falla.
3. **Mezclar tres.** Con tres listas ordenadas de n nodos cada una, compare mezclarlas de a dos contra escribir una mezcla de tres cabezas. ¿Cuántas comparaciones hace cada una?
4. **Merge sort sobre la doble.** ¿Qué hay que arreglar al terminar la mezcla para que los enlaces `anterior` queden bien? ¿Cambia el costo?
5. **El nodo centinela.** Un nodo fijo que no guarda elementos y que siempre está al frente. ¿Cuáles de los casos de `insertar` y `eliminar` desaparecen?

Resumen I

- El campo `anterior` compra desenlazar un nodo en $\Theta(1)$ y recorrer hacia atrás; cobra un puntero más por nodo y dos enlaces que mantener en cada modificación.
- Cerrar el círculo borra los `NULL` de los extremos: `insertar` pasa de cuatro casos a dos y el último se alcanza en un paso. Queda el caso del nodo único, que se apunta a sí mismo.
- Con el contador y el enlace hacia atrás se camina por el lado más corto: el peor caso baja de $n-1$ a $\lceil n/2 \rceil$ pasos y la cota sigue siendo $\Theta(n)$.
- Merge sort parte con dos punteros a distinta velocidad, ordena cada mitad y mezcla empalmando nodos: $\Theta(n \log n)$ en tiempo, $\Theta(1)$ de espacio en la mezcla y $\Theta(\log n)$ de pila. Sobre un arreglo la mezcla pide $\Theta(n)$ auxiliar.

Resumen II

Lo que sigue

Pila y cola por dentro: qué implementación conviene a cada una cuando las operaciones son solo por los extremos, y por qué una cola sobre arreglo necesita dar la vuelta.

Referencias I


R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 6, listas enlazadas: doblemente enlazadas y circulares.


T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Sección 2.3, merge sort y el análisis de la recurrencia; sección 10.2, listas enlazadas.


N. Kalicharan. *Data Structures in C*. 2008. Listas enlazadas y sus variantes.